PFP I: Exact Graph Repair and the Boundary Between a Proposal and a Certificate

Jeremy H. Carroll

2026-10-02 · Version 0.1.0 · Research paper · Author-review revision

Code baseline: c819bec432446dd3c9b1edf1583384cfe76c6c71

Abstract

A proposed solution and a checked solution are different objects. The Proposal Fidelity Protocol (PFP), developed in the repository under the legacy name Symbol Game, makes this difference explicit: a proposer supplies a candidate, while a host checks a declared mathematical predicate before changing accepted state. We develop a small, fully specified example using inconsistent differences on a finite graph. Weighted absolute residuals measure the quality of a proposed potential; a separate quotient minimum measures the inconsistency of the fixed edge data. We derive their relationship, give matching primal and dual witnesses, prove the weighted cycle formula, and explain sparsity and uniqueness with their necessary hypotheses. We also show why strict rational descent alone does not ensure termination. The contribution is a reproducible protocol and exposition of established finite optimization facts, with an explicit account of what the current software returns. A paper-local Python reference builds and checks the full primal and dual certificate on small graphs. The native Rust solver returns only a minimum and a minimizing potential after bounded exhaustive search. No model accuracy improvement or general reasoning guarantee is inferred.

1. A problem that a reader can check

Readers beginning with symbols and packets can first work through Paper 000, especially its typed-prompt and returned-packet notebook. This paper strengthens that finite proposal/check distinction: a candidate potential has a checkable cost, while a separate lower-bound witness is needed to certify that the cost is optimal.

Suppose three instruments report relative offsets. The first says that b is one unit above a; the second says that c is one unit above b; the third says that a is one unit above c. Walking around the triangle would raise a level by three units and return to its starting point. These three reports cannot all be differences of one assignment of levels.

Write the directed edges as $a \to b, b \to c, c \to a$, the reports as $c = (1, 1, 1)$, and the penalties for disagreeing with them as $w = (1, 2, 3)$. A proposal consists of three levels $s = (s_a, s_b, s_c)$. Its cost is

$$D_c(s) = |1 - (s_b - s_a)| + 2|1 - (s_c - s_b)| + 3|1 - (s_a - s_c)|.$$

The all-zero proposal costs six. The proposal $s = (0, -2, -1)$ has residuals $(3, 0, 0)$ and costs three. A host can verify both calculations without trusting the proposer's explanation. To establish that three is the best possible value, however, it needs an additional argument. That argument will be a circulation: one unit of flow around the cycle gives an independent lower bound of three.

This example has practical interpretations in clock offsets, relative rankings, and consistency checks on a ledger. Such interpretations require a separate decision about what the numbers mean and

which reports may be corrected. The optimization problem itself cannot decide whether an instrument is wrong.

PFP uses this small problem to expose three distinct questions: whether a candidate is well formed, whether it improves an accepted state, and whether it certifies a mathematical conclusion. A language model, a search routine, or a person may generate the candidate. The checking rule is the same.

2. The finite model and its two costs

Let $G = (V, E)$ be a finite graph with a chosen orientation, rational edge reports c_e, and positive rational penalties w_e. Let B be its edge-by-vertex incidence matrix, so $(Bs)_{u\to v} = s_v - s_u$. We allow a disconnected graph; potentials have one irrelevant additive constant on each component. Both implementations described in Section 6 exclude self-loops.

The words *0-cochain* and *1-cochain* denote assignments to vertices and edges. No continuum geometry is required. The current graph problem has no filled faces: $C^2 = 0$. It therefore does not measure a nonzero face curvature.

For a proposed potential s, define

$$r = c - Bs, \qquad D_c(s) = \sum_e w_e |r_e|, \qquad \Phi(c) = \min_f D_c(f).$$

The first quantity depends on the proposal. The second is the least achievable cost for the fixed reports. Both are nonnegative, and $D_c(s) \geq \Phi(c)$. A rational minimum exists: the problem is a feasible, bounded rational linear program after splitting the absolute values and fixing one potential per component.

Exactness. $\Phi(c) = 0$ precisely when $c = Bf$ for some potential f. The forward implication uses attainment and positivity of every penalty: a zero weighted sum of absolute residuals forces every residual to vanish. The reverse implication follows by substitution. If a penalty were zero, a nonzero residual on that edge could become invisible.

Invariance. For every potential h,

$$\Phi(c + Bh) = \Phi(c).$$

Indeed, substituting $f' = f - h$ gives the same set of residuals in the minimization. Changing a proposed section s while holding c fixed also leaves $\Phi(c)$ unchanged. Thus a section update can reduce $D_c(s)$ toward the obstruction $\Phi(c)$, but cannot erase that obstruction.

Some historical APIs instead replace the edge reports by the fitted differences Bs. That operation changes the question. We call it *report replacement* here. Its new quotient cost is zero by construction, but the original reports have not thereby become consistent. This distinction is essential when importing the calculation into a scientific application.

3. A certificate for optimality

A vector y on the edges is a feasible dual circulation when

$$B^{\mathsf{T}} y = 0, \qquad |y_e| \le w_e.$$

The first condition says that flow balances at every vertex. For every potential s,

$$c^{\mathsf{T}} y = (c - Bs)^{\mathsf{T}} y \le \sum_e |c_e - (Bs)_e|\, |y_e| \le D_c(s).$$

This is weak duality, proved directly by balance and the triangle inequality. Consequently a proposed potential $s^\star$ and feasible circulation $y^\star$ certify optimality whenever

$$D_c(s^\star) = c^{\mathsf{T}} y^\star.$$

Checking this pair requires exact arithmetic, coverage of the declared vertices and edges, balance, capacity bounds, and equality of objectives. Equality alone is insufficient. A vector with a flow leak or excessive capacity is not a lower-bound certificate, even if its dot product happens to match the primal cost.

Linear-programming strong duality gives the full characterization

$$\Phi(c) = \max_{B^{\mathsf{T}} y = 0,\ |y_e| \le w_e} c^{\mathsf{T}} y.$$

One standard primal formulation is $Bs + r^+ - r^- = c$, with $r^+, r^- \ge 0$, minimizing $w^{\mathsf{T}}(r^+ + r^-)$. Splitting free potentials gives a standard finite rational LP. This is an application of established optimization theory, not a new duality theorem. The derivation also fixes the sign convention: a maximizing circulation matches $w_e \operatorname{sign}(r_e)$ on nonzero residuals.

For the introductory triangle, $y = (1, 1, 1)$ balances and respects the capacities $(1, 2, 3)$. Its objective is $1+1+1 = 3$. The potential $(0, -2, -1)$ also costs three. These two elementary calculations prove the claimed minimum without appealing to a solver's status label.

4. Cycles, sparsity, and uniqueness

Consider a directed cycle of $n \ge 3$ edges. Put $h = \sum_e c_e$ and $\rho = \min_e w_e$. Telescoping gives $\sum_e r_e = h$, hence

$$\sum_e w_e |r_e| \ge \rho \sum_e |r_e| \ge \rho |h|.$$

Choose a least-penalized edge j, set $r_j = h$, and set every other residual to zero. The edge vector $c - r$ has sum zero, so successive partial sums construct a closing potential. This attains the lower bound:

$$\Phi(c) = \rho |h|.$$

If $h \neq 0$ and the cheapest edge is unique, equality forces every residual onto that edge. The fitted edge vector is unique; the potentials still have their additive-constant freedom. If several edges share

the cheapest penalty, same-sign splits of h among them are equally good. With uniform weights and nonzero h on an n-cycle, the optimal residuals form an $(n-1)$-dimensional simplex. The cycle rank is one, so cycle rank does not bound this optimal-face dimension.

For a general graph, there is *some* optimal residual supported on at most $m = |E| - \operatorname{rank} B$ edges. To see this, impose the coset condition $r \in c + \operatorname{im} B$ using m independent linear equations. Split $r = r^+ - r^-$ and take a basic optimal solution. It has at most m nonzero split variables. Both signs of one coordinate cannot be positive at an optimum, because reducing both lowers the positive-weight objective. This establishes in this model an existence statement about a sparse optimum. Other optima may be dense.

A useful sufficient uniqueness condition is the weighted null-space property. For a residual supported on S, require every nonzero $z \in \operatorname{im} B$ to satisfy

$$\sum_{e \in S} w_e |z_e| < \sum_{e \notin S} w_e |z_e|.$$

The triangle inequality then gives

$$\|r + z\|_{1,w} \geq \|r\|_{1,w} - \|z_S\|_{1,w} + \|z_{S^c}\|_{1,w} > \|r\|_{1,w}.$$

The residual is therefore the unique optimum in its coset. This condition is sufficient; failure of it is not a general proof that a particular optimum is nonunique. A deterministic tie rule only selects among equal optima.

These facts do not imply that an LLM's token probabilities become sparse. For a probability vector p, $\|p\|_1 = 1$. Graph residual minimization, a decoder mask, and rejecting an invalid response are different operations.

5. From mathematics to a proposal protocol

A minimal PFP run pins the input graph, reports, penalties, allowed moves, checker version, and fuel. It stores an accepted potential and a record of decisions. A candidate never edits accepted state directly.

For this section we define a strictly improving admission rule: an admitted section update must parse, cover the required carrier, and strictly reduce D_c under the fixed reports. An executable that allows stationary updates has a different declared admission rule; the strict-descent statement does not describe that policy. A refused candidate leaves the accepted mathematical state unchanged. A terminal exactness claim needs zero residuals; a terminal optimality claim needs the declared optimization evidence, ideally the primal and dual certificate above. An obstruction is a legitimate answer when supported by the appropriate witness.

Monotonicity applies to this fixed-data update rule. Adding an inconsistent report can raise the cost. Restoring an earlier checkpoint can also restore a higher cost. Neither action should be included in a theorem saying that every admitted action decreases one number. Changes of input or rollback must remain visible in the record.

Strict descent over the rationals is not a termination argument. On a single edge with report zero and unit penalty, fix $s_a = 0$ and propose $s_b = 2^{-t}$. The costs decrease forever and never equal

zero. A practical run instead consumes one unit of finite fuel per attempted transition and stops at certification or an explicit unfinished result. This bounds attempts; a separate timeout is needed to bound an individual parser, solver, or external call.

Hash chaining helps detect changes relative to a retained trusted root. It does not prove a mathematical claim, make a log physically immutable, or authenticate a provider by itself. Local mathematical acceptance confers no external execution permission; the repository records that policy separately from the arithmetic result.

6. What the implementation actually supplies

The package contains two exact implementations of the graph problem and three small Lean files. Their scopes differ.

The native solver is graph_phi1.rs. It parses exact rationals written as integers or fractions, and it rejects decimal strings, duplicate vertices, unknown endpoints, self-edges, and nonpositive weights. It enumerates spanning-forest potentials and compares their exact weighted costs. Search is capped at 100,000 forests. Exceeding the cap returns `EnumerationLimit`; it does not return a partial search as an optimum.

The spanning-forest construction is justified by the hyperplane arrangement $c_e - (Bs)_e = 0$. After removing constant-potential freedom, its normals span the component potential space. The convex piecewise-linear objective has an optimum at an arrangement vertex, and independent zero-residual edges at a vertex form a spanning forest. Enumerating these vertices therefore supplies a finite exact algorithm. It can be exponential in graph size. The existence of polynomial LP algorithms does not make this implementation polynomial.

The native response contains `phi1`, `gauge`, and `forests_checked`. It does not contain a dual circulation. The compiled program prints one JSON object: status PASS with these three fields, or status REFUSE with a reason, in which case it exits with status 2. The Python adapter passes exact rationals to that executable. On a PASS object it accepts `phi1` and gauge entries only as integer or fraction strings, requires the gauge to name exactly the declared vertices, requires `forests_checked` to be an integer from 1 through 100,000, rejects a negative `phi1`, and requires the reported potential's weighted absolute cost to equal `phi1`. That cost check is consistency of the returned primal pair. It is not a dual lower bound, and the native enumeration stays trusted. A JSON array, a non-object gauge, a float or Boolean, a decimal string, a missing forest count, or a cost mismatch raises `NativeFunctionalProtocolError`. A native refusal, timeout, or nonzero exit still raises its own typed error. The adapter does not substitute another solver's answer for any of those failures. It finds the executable through an explicitly configured path or the crate's own build directory. It does not hash or pin the binary, so a reproducible source-to-binary build needs additional custody. Only when no executable exists, and the caller has not required the native path, does the adapter answer with the Python reference described next.

The paper-local Python reference computes the same minimum and also the certificate of Section 3. For each connected component it enumerates spanning trees, capped at 100,000 per component; exceeding the cap raises `SpanningTreeLimitExceeded` instead of returning a partial minimum. Among minimizing trees it builds a balanced circulation within the capacities. It returns the potential and the circulation only after an independent verifier has checked exact types, coverage of the declared vertices and edges, balance, capacities, and equality of the two objectives. The governor built on this reference applies the strict-descent rule of Section 5: it admits a potential update only when the cost strictly decreases, and it computes the minimum with the reference rather than the native

executable. An admitted update with cost zero is labeled EXACT. An admitted update whose positive cost equals the quotient minimum is labeled HOLD, the same label VERIFY returns on that state. Proposal numbers are integers, `Fraction` values, or integer and fraction strings. Floats, Booleans, and decimal strings are refused and leave the accepted state unchanged. The governor has no fuel counter. Demo 1 owns the finite descent loop used in the worked examples, and the adapter owns a separate subprocess timeout. Like the native search, the reference is exponential in graph size and is meant for small examples.

The local Lean files also have limited statements. OddCycleObstruction.lean formalizes that no assignment of signs ± 1 to the vertices of a triangle makes all three edge products equal to -1. InvolutionLaws.lean formalizes that a configuration and its image under a supplied sign-reversing, weight-preserving map have signed natural-number weights summing to zero. NonAuthority.lean encodes a declared policy as a type with a single value, so its theorem holds by construction and checks no runtime behavior. The files build with Lean 4.34.0-rc1. The axiom report lists propext and Quot.sound for the pair-cancellation theorem and no axioms for the other two. None establishes the general graph LP, a production runtime, or termination of arbitrary rational descent.

7. Reproduction and adversarial controls

The commands below run from this package root. They use `py -3.12`, the Windows launcher for Python 3.12; on other systems, substitute a Python 3.12 interpreter. Test files are run one per command.

```
# Native Rust solver: unit tests, then the command-line program
cargo test -j 1 --manifest-path code/rust/Cargo.toml
cargo build -j 1 --manifest-path code/rust/Cargo.toml --bin solve_graph_phi1

# Python tests
py -3.12 -m pytest code/tests/test_manuscript_witnesses.py -q
py -3.12 -m pytest code/tests/test_review_regressions.py -q
py -3.12 -m pytest code/tests/test_zenodo_regression_suite.py -q
py -3.12 -m pytest code/tests/test_governor.py -q
py -3.12 -m pytest code/tests/test_certificate_gate.py -q
py -3.12 -m pytest code/tests/test_exact_fock_gap.py -q
py -3.12 -m pytest code/tests/test_fock_path_proposition.py -q
py -3.12 -m pytest code/tests/test_proposed_boundaries.py -q

# Offline demonstrations (Demos 1 to 3) and the package figure
py -3.12 code/run_demos.py --all
py -3.12 code/generate_figures.py

# Lean statements
cd lean4
lake build
```

The live debate demo is optional and is not part of this paper's evidence. It calls the OpenRouter service and requires the reader's own OpenRouter key, supplied through the environment or through a file named on the command line. The file .env.example holds a placeholder only. For example:

```
py -3.12 code/run_demos.py --demo 3 --live --output-dir <new-directory> --env-file .env
```

See README.md for setup.

A minimal independent mathematical replay should check the triangle's primal cost six and three, dual value three, and exactness after changing the third report to -2. It should reject a dual flow with one entry deleted, an over-capacity circulation, an unknown endpoint, and a zero penalty. A tied uniform cycle should admit multiple optimal residuals; a rational descending sequence should exhaust fuel rather than be called convergent. Demo 1 performs each of these checks in exact rational arithmetic. The package figure fig1_descent_and_obstruction.png plots the triangle descent and the single-edge sequence from exact governor runs.

The claim ledger separates prose proofs, executable checks, and open implementation obligations. Historical receipts remain in results/, and the contract files of the certificate gate used by Demo 2 remain in specs/; their source bindings and dates must be checked before reuse. Signed cancellation and the exact Fock example are developed in the companion paper PFP II (Carroll, 2026a; see docs/references.md). This package retains the exact Fock checker and its tests.

8. Related work, contribution, and limits

Necula's *Proof-Carrying Code* (Necula, 1997) already separates untrusted production from consumer-side checking against a declared policy. PFP applies this established design idea to proposed reasoning moves and finite optimization witnesses. It does not claim to originate independent verification.

The LP duality used here is standard; Boyd and Vandenberghe's *Convex Optimization* (Boyd and Vandenberghe, 2004) supplies a systematic treatment of primal and dual bounds. Our contribution is the graph-specific derivation, exact examples, and mapping to a bounded implementation.

The protocol is useful when a problem has an explicit checker and candidate search is less trustworthy than verification. Its conclusions are conditional on correct input translation, a correct checker, preserved arithmetic assumptions, and reliable execution. A faulty checker can consistently accept a wrong claim. A correct graph certificate says nothing about unmodeled physics or the quality of natural-language reasoning. Establishing model benefit requires a controlled comparison such as the one proposed, but not yet run, in the companion paper PFP III (Carroll, 2026b; see docs/references.md).

Verification for this revision

The package verifier (`verify_deposit.py`) checks the manifest and the two-way file census; `tools/rebuild_release.py --check` checks the release archive. The complete package test suite, code lint, Lean build, notebook execution, and Rust cargo tests are recorded in docs/release_validation_2026_10_02_review_repairs.json. That receipt binds the repaired adapter (code/native_functional.py) and governor (code/symbol_game_governor.py) by SHA-256, and those hashes match the current files. Follow-on targeted checks (Demo 1 string potentials, BQNF route aliases, and worksheet verification) are recorded in docs/release_validation_2026_10_02_continued_review.json. Each receipt lists its specific commands, tool versions, exit statuses, and execution scope. The code baseline c819bec432446dd3c9b1edf1583384cfe76c6c71 names the cited source before the 2026-10-02 repairs to the adapter and the governor; that commit does not identify the repaired bytes. The Rust unit tests run with the `cargo test` command of Section 7 (using a shortened target path when link.exe path limits apply). Earlier exploratory receipts (docs/release_validation_2026_10_01.json and docs/release_validation_2026_10_02.json) retain their original dates and scope.

References

- Boyd, Stephen, and Lieven Vandenberghe (2004). *Convex Optimization*. Cambridge University Press. doi:10.1017/CBO9780511804441; author-hosted book.
- Carroll, Jeremy H. (2026a). *PFP II: Signed Certificates, Resumable Search, and a Restricted Path-Pairing Obstruction*. Version 1.1.0, author-review preprint, Paper 002 in this series. Repository path omega_engine_atlas/coda_atlas/omega_engine_symbol_game/002_the_symbol_game_signed_presentations_and_search_frontier.
- Carroll, Jeremy H. (2026b). *PFP III: Admitting Failure Detectors and Testing Their Effects*. Version 1.1.0, author-review preprint, Paper 003 in this series. Repository path omega_engine_atlas/coda_atlas/omega_engine_symbol_game/003_the_symbol_game_host_admitted_failure_detectors.
- Necula, George C. (1997). "Proof-carrying code." *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '97)*, pp. 106–119. ACM Press. doi:10.1145/263699.263712; author's publication record.